Rust: Segurança na Programação

Structs e Métodos

Nuno Neto — 2026

CONNECT5
EDIH European Digital Innovation Hubs Network
U.PORTO FACULDADE DE CIÊNCIAS UNIVERSIDADE DO PORTO
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
PRR Plano de Recuperação e Resiliência

Structs em Rust

Para além dos tipos escalares e compostos, Rust permite ao desenvolvedor definir os seus próprios tipos — os **structs**.

São tipos compostos que agrupam e *estruturam* vários valores à nossa escolha.

Structs em C

Structs/Classes em C++

Classes em Java/Python

Túpulos em Rust

Os Structs do Rust são uma mistura das 4 implementações com toques particulares.

Características dos Structs

Sem Herança

Não podem ser abstratos. Encorajam **composição** e **polimorfismo baseado em traits**.

Campos Visíveis

Todos os campos são automaticamente visíveis ao módulo onde o struct é definido.

Sem Construtor

Todos os campos **têm que estar definidos** em **todos os momentos**.

Layout Simples

Layouts de memória simples, facilitando a integração com o modelo de **ownership**.

Sintaxe: Declaração e Inicialização

Declaração

Usa-se a keyword **struct** seguida de chavetas. Cada membro segue a forma nome_var: tipo_var,

```
struct Test {
  test: String,
  test_1: i32,
  test_2: u64
}
```

Inicialização

Todos os campos têm de ser atribuídos. Suporta *initialization shorthands* quando o nome das variáveis coincide.

```
let test = Test {
 test: String::from("Ola"),
 test_1: 10,
 test_2: 20
};

// Shorthand
let test_struct = Test {
 test, test_1, test_2
};
```

Acessos e Ownership

Dot Notation

Para aceder a elementos usa-se a **dot notation**: instancia.campo

Tipos Copy

O valor é **copiado** — objeto independente.

Tipos não-Copy

A instância inteira é **consumida**. Os restantes valores são descartados.

Referências

Usar &test.campo para aceder **sem mover** — a instância permanece válida.

Modificar, Desconstruir e Atualizar

1

Modificar

Toda a instância tem de ser **mut**. Usa-se dot notation para atribuir novos valores.

```
let mut test = Test { ... };
test.test =
String::from("Ola mundo");
```

2

Desconstruir

Obtém todas (ou um subconjunto) as partes do struct. O **objeto original deixa de ser válido**.

```
let Test { test: valor_test,
  test_1: valor_test_1, .. }
= test;
```

3

Struct Update Syntax

Instanciar com apenas alguns valores diferentes de um struct existente.

```
let test2 = Test {
 test: String::from("Teste
2"),
 ..existing_struct
};
```

Variantes de Structs

Struct Normal

Campos com nome e tipo. A forma mais comum.

```
struct Ponto {
 x: i32,
 y: i32
}
```

Tuple Struct

Elementos sem nome, apenas tipos. Útil para diferenciar tuplos semelhantes.

```
struct Ponto3D(i32, i32, i32);
struct CorRGB(i32, i32, i32);
```

Unit Struct

Sem membros. Usado como tipo auxiliar para implementações de *traits* sem dados associados.

```
struct UnitStruct;
```

Métodos e Funções Associadas

Métodos

Definidos num bloco **impl**, o primeiro parâmetro é sempre self (ou &self, &mut self).

```
impl Rectangulo {
  fn area(&self) -> u32 {
    self.altura * self.largura
  }
}
```

Chamada: instancia.metodo() equivale a Struct::metodo(instancia)

Funções Associadas

Primeiro argumento **não é self**. Usadas tipicamente como construtores idiomáticos.

```
impl Test {
 fn new() -> Self {
 Self {
 test: String::from("test")
 }
 }
}
```

Leaking *this* — Um Problema que Rust Não Tem

O Problema (C++, Java...)

Durante a inicialização, é possível usar this — uma referência ao objeto ainda não construído — levando a acessos indevidos e comportamento indefinido.

A Solução do Rust

Em Rust, não existem construtores. Um objeto **existe ou não existe** — sem intermédios. Referências a objetos incompletos são **impossíveis**.

Derivações

Structs em Rust permitem compor características dos seus membros através de **derivações**, sem implementação manual adicional.

Usa-se a macro #[derive()] aplicada ao struct:

```
#[derive(Debug, Eq, PartialEq, Clone)]
struct Rectangulo {
  altura: u32,
  largura: u32,
}
```

Copy / Clone

Duplicação de valores

Debug

Formatação para depuração

Eq / Ord

Comparação e ordenação